

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание 4

Технический дизайн микросервисной архитектуры

Выполнил:
Катков Алексей Сергеевич
Группа К3339

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Разделить монолитное бэкэнд-приложение на микросервисы, придерживаясь концепции database-per-service. Спроектировать взаимодействие между микросервисами, разделение баз данных и OpenAPI эндпоинты для межсервисного взаимодействия.

Ход работы

1. Описание исходного монолитного приложения

В качестве исходного приложения использовался сервис обмена рецептами, разработанный на базе Express.js, TypeORM и PostgreSQL.

Приложение поддерживает следующие возможности:

- регистрация и авторизация пользователей;
- работа с JWT токенами;
- создание рецептов;
- добавление ингредиентов;
- добавление типов блюд;
- добавление шагов приготовления;
- комментарии;
- лайки;
- сохранение рецептов;
- подписки между пользователями.

В монолитном приложении использовались следующие сущности:

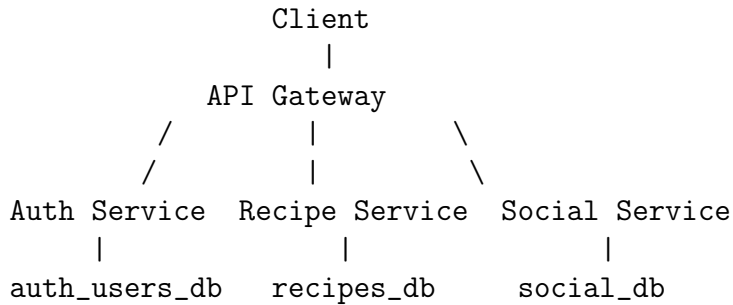
- User
- Recipe
- Ingredient
- DishType
- RecipeStep
- Comment
- Like
- SavedRecipe
- Follow

Все сущности использовали одну общую базу данных PostgreSQL.

2. Выделение микросервисов

Монолитное приложение было разделено на несколько независимых микросервисов.

2.1 Архитектурная схема



API Gateway используется как единая точка входа для клиентского приложения.

2.2 Auth/User Service

Сервис отвечает за:

- регистрацию пользователей;
- авторизацию;
- генерацию JWT токенов;
- хранение профилей пользователей;
- подписки между пользователями.

Сущности:

- User
- Follow

2.3 Recipe Service

Сервис отвечает за:

- создание рецептов;
- ингредиенты;
- типы блюд;
- шаги приготовления.

Сущности:

- Recipe
- Ingredient
- DishType
- RecipeStep

2.4 Social Service

Сервис отвечает за:

- комментарии;
- лайки;
- сохраненные рецепты.

Сущности:

- Comment
- Like
- SavedRecipe

2.5 API Gateway

API Gateway является единой точкой входа для клиента.

Основные функции API Gateway:

- маршрутизация запросов;
- проверка JWT токена;
- агрегация данных;
- перенаправление запросов в нужный сервис;
- скрывание внутренней структуры микросервисов от клиента.

3. Распределение сущностей по сервисам

Сервис	Сущности
Auth/User Service	User, Follow
Recipe Service	Recipe, Ingredient, DishType, RecipeStep
Social Service	Comment, Like, SavedRecipe

4. Разделение баз данных

Для реализации подхода database-per-service были выделены отдельные базы данных.

4.1 Auth/User Database

База данных хранит:

- пользователей;
- подписки.

Название базы данных:

<code>auth_users_db</code>

4.2 Recipe Database

База данных хранит:

- рецепты;
- ингредиенты;
- типы блюд;
- шаги приготовления.

Название базы данных:

```
recipes_db
```

4.3 Social Database

База данных хранит:

- комментарии;
- лайки;
- сохраненные рецепты.

Название базы данных:

```
social_db
```

В микросервисной архитектуре прямые foreign key между базами данных разных сервисов не используются. Вместо этого сервисы взаимодействуют друг с другом через HTTP REST API.

5. Взаимодействие микросервисов

Микросервисы взаимодействуют друг с другом через HTTP REST API.

5.1 Взаимодействие Recipe Service и Auth Service

Recipe Service хранит только идентификатор автора рецепта:

```
authorId
```

Для получения информации об авторе используется запрос:

```
GET /internal/users/{id}
```

5.2 Взаимодействие Social Service и Recipe Service

Перед созданием комментария или лайка Social Service проверяет существование рецепта.

```
GET /internal/recipes/{id}
```

5.3 Межсервисная авторизация

Для защиты internal endpoint используются service-to-service JWT токены.

Только доверенные сервисы могут выполнять запросы к внутренним endpoint.

5.4 Взаимодействие через API Gateway

Клиент отправляет запрос в API Gateway.

API Gateway:

- проверяет JWT токен;
- определяет нужный сервис;
- перенаправляет запрос;
- возвращает объединенный ответ клиенту.

6. OpenAPI эндпоинты

6.1 Auth Service

```
POST /auth/register
POST /auth/login
GET /users/me
GET /internal/users/{id}
```

6.2 Recipe Service

```
GET /recipes
POST /recipes

PATCH /recipes/update?id=1
DELETE /recipes/delete?id=1

GET /ingredients
POST /ingredients

GET /dish-types
POST /dish-types

GET /recipe-steps
POST /recipe-steps
```

6.3 Social Service

```
GET /comments
POST /comments

POST /likes
DELETE /likes

POST /saved-recipes
DELETE /saved-recipes
```

7. Примеры запросов и ответов

7.1 Создание рецепта

Запрос:

```
POST /recipes
```

Тело запроса:

```
{
  "title": "Pasta",
  "description": "Cheese pasta",
  "difficulty": "easy",
  "cookingTime": 20,
  "servings": 2,
  "authorId": 1,
  "ingredientIds": [1, 2],
  "dishTypeIds": [1]
}
```

Ответ:

```
{
  "id": 5,
  "title": "Pasta"
}
```

7.2 Ошибка авторизации

Ответ:

```
{
  "message": "Forbidden: token is invalid or expired"
}
```

8. Возможные ошибки

В процессе проектирования были выделены следующие возможные проблемы:

- потеря связи между сервисами;

- проблемы синхронизации данных;
- увеличение задержек из-за сетевого взаимодействия;
- необходимость централизованной авторизации;
- сложность поддержки нескольких баз данных;
- необходимость мониторинга состояния сервисов.

9. План перехода от монолита к микросервисам

Переход предлагается выполнять поэтапно:

1. выделение Auth Service;
2. перенос Recipe Service;
3. перенос Social Service;
4. добавление API Gateway;
5. разделение общей базы данных;
6. настройка межсервисного взаимодействия;
7. тестирование взаимодействия сервисов.

Вывод

В результате работы была спроектирована микросервисная архитектура для сервиса обмена рецептами.

Монолитное приложение было разделено на несколько независимых микросервисов с отдельными базами данных. Были определены способы взаимодействия между сервисами, спроектированы REST API эндпоинты и описаны возможные проблемы при переходе на микросервисную архитектуру.

Использование микросервисной архитектуры позволяет упростить масштабирование, поддержку и независимое развитие отдельных компонентов системы.